The magic of asyncio explained (hackernoon.com)

168 points by apoorvgarg on Aug 8, 2018 | hide | past | favorite | 89 comments

anilakar on Aug 8, 2018 | next [–]

Yet another asyncio tutorial that shows you to run a few sleep tasks concurrently. Can we finally get one that shows how to do *real* stuff such like socket programming, wrapping non-async-compatible libraries and separating cpu-intensive blocking tasks to awaitable threads?

sametmax on Aug 8, 2018 | parent | next [–]

> such like socket programming

That's one of my biggest pet peeves (and if you see my other comments, you'll notice I have quite a few).

To do socket programming in asyncio, you can either use:

- protocols, with a nice reusable API and an interface that clearly tells you where to do what. But you can't use "await". You are back to creating futures and attaching callback like 10 years ago.

- streams, where you can use async / await, you but get to write the entire life cycle by yourself all over again.

I get that protocols are faster, and match Twisted model, and I get that streams are pure and functional, but none of this is easy. I use Python to make my life easier. If I wanted extreme perfs I'd use C. If I wanted extreme pureness I'd use Haskell.

> wrapping non-async-compatible libraries and separating cpu-intensive blocking tasks to awaitable threads

That's the one of the things asyncio did right. Executors are incredibly simple to use, robust and well integrated.

Problem is: they are badly documented and the API is awkward.

I won't write a tutorial in HN, but as a starting point:

You can use:

```
loop = asyncio.get_event_loop()
future = loop.run_in_executor(executor, callback, arg1, arg2, arg2...)
await future
```

If you pass "None" as an executor, it will get the default one, which will run your callback in a thread pool. Very useful for stuff like database calls.

But if you want CPU intensive task, you need to create an instance of ProcessPoolExecutor, and pass it to run_in_executor().

I say it's one of the things asyncio did right because the pools not only distribute automatically the callbacks among the workers of the pool (which you can control the number), but you also get a future back which you can await transparently.

mtrovo on Aug 8, 2018 | root | parent | next [–]

> Problem is: they are badly documented and the API is awkward.

That's my main problem with asyncio right now, bumping into problems and trying to find how to fix them by looking into the documentation is rather difficult. The documentation right now feels more like a documentation for an unsupported old library.

Also it's awkward that you cannot resolve kwargs on the functions that you pass to asyncio, like the callback in run_in_executor. You have to wrap the function in a partial resolving all kwargs and then send it to the executor.

windexh8er on Aug 8, 2018 | root | parent | prev | next [–]

I'm curious of your take on Trio or Curio. Do either address the peeves you've outlined?

https://github.com/python-trio/trio https://github.com/dabeaz/curio

sametmax on Aug 8, 2018 | root | parent | next [–]

Trio is a better Curio, so you don't really need Curio anymore. It was what started everything and deserves credit for that though.

As for Trio, it's what asyncio should have been from the beginning, at least for the high level part (although not for the pet peeves of socket programming: it's too low level for Python IMO)

The problem with Trio is that it's incompatible with asyncio (minus some verbose quirky bridges), so you get yet another island, yet another ecosystem. So what, now we get twisted, tornado, gevent, qt, asyncio... and trio ?

The madness must stop.

And that's why I think there is a better way: creating a higher level API for asyncio, which enforces the best practices and make the common things easy, and the hard things decent.

A complete rewrite like Trio would be better (e.g: it famously handles Ctrl + C way better and has a smaller API). But this ship has sailed. We have asyncio now.

asyncio is pretty good honestly. But it needs some love.

So, considering asyncio is what we have to work with, and by experience, it's quite great if you know what you are doing, I advice people to actually write a wrapper around it.

If you don't feel like writing a wrapper, I'll plug in the one I'm working on in case people are curious: https://github.com/Tygs/ayo

It:

- is based on asyncio. Not a new framework. Integrated transparently with normal asyncio.

- implement some lessons leaned from trio (e.g: nurseries, cancellation, etc)

- expose a sweet API (running blocking code is run.aside(callback, *args,* *kwargs), and automate stuff it can (@ayo.run_as_main setup the event loop and run the function in one row)

- make hard things decent: timeout and concurrency limit are just a param away

I does need some serious doc, including a rich tutorial which features a pure asyncio part. Also it needs some mix between streams and protocols. I'm not going to skip that part, I think it's mandatory, but I'll need many months to make the whole thing.

Now, I am not Nathaniel or Yury, so my work is not nearly as bullet proof as theirs. I would not advice to install ayo in prod now, but I think it's a great proof of concept of how good asyncio can be.

And we most certainly can do even better.

zeckalpha on Aug 8, 2018 | root | parent | prev | next [–]

> If I wanted extreme perfs I'd use C. If I wanted extreme pureness I'd use Haskell.

Haskell concurrent socketry is decent:

https://kyle.marek-spartz.org/posts/2014-08-26-concurrent-im...

diftraku on Aug 8, 2018 | root | parent | prev | next [–]

It's hilarious when a single comment on HN opens asyncio more than the tutorial being discussed.

sametmax on Aug 8, 2018 | root | parent | next [–]

Well, honestly, I think most HN readers go to the comments before the actual article.

I know I do.

The whole value of this website is that we got 1000 of experts in their fields, ready to give you their insight.

philwelch on Aug 8, 2018 | root | parent | next [–]

HN isn't what it was a few years ago, but it's still a hell of a lot better than Hackernoon.

ddevault on Aug 8, 2018 | root | parent | prev | next [–]

Have you considered contributing improvements to the documentation, or even the API? Python is an open source project.

sametmax on Aug 8, 2018 | root | parent | next [–]

Yes. If you are not part of the club, it takes approximatly 18 months from a post on python-idea to an implementation, after so much debate it's madening. And most of the time gets rejected.

The process of contributing to python is more frustrating than writing for wikipedia.

Much easier to write something in pypi, then come back to python-idea once it gets popular.

ddevault on Aug 8, 2018 | root | parent | next [–]

Well, with doc updates I think you'll probably find the process much smoother, and it's a good way of getting trust among the "club".

sametmax on Aug 8, 2018 | root | parent | next [–]

That's fair. I'll get in touch with stinner on the next pycon, I think he is the guy for that.

ddevault on Aug 8, 2018 | parent | prev | next [–]

If anyone wants to see some small, practical asyncio code in action, here's a little LMTP daemon I wrote recently:

https://git.sr.ht/~sircmpwn/lists.sr.ht/tree/lists-srht-lmtp via https://git.sr.ht/~sircmpwn/lists.sr.ht

Or getting deeper, another project which implements Synapse's[0] RPC protocol and encapsulates high-level RPC actions in asyncio sugar:

https://git.sr.ht/~sircmpwn/broca/tree/broca/connection.py

Code which uses this code:

https://git.sr.ht/~sircmpwn/broca/tree/broca/rpc.py

[0] https://synapse-bt.org

AboutTheWhisles on Aug 8, 2018 | parent | prev | next [–]

This architecture is made to match software threads to (logical) hardware threads, then have them loop over data separated into chunks that don't depend on each other.

If a function is blocking, needs the CPU and isn't thread safe, it can be wrapped in a message passing node that will get skipped over if a thread is already running it.

Every separate chunk of data that it creates will be dealt with concurrently and the high level structure can be put together in a graph that uses openGL.

https://github.com/LiveAsynchronousVisualizedArchitecture/la...

luckycharms810 on Aug 8, 2018 | parent | prev | next [–]

It is not in the style of a blog, but I put this together a few years ago to teach people how async programming works. It includes a handful of examples on generators, Coroutines, how to make use of the socket libs non blocking functionality, and how to tie them together to get the yield syntax folks are used to.

It doesn’t use any asyncio parts of python but is just meant to show what’s happening under the hood.

https://github.com/ltavag/async_presentation?files=1

pmlnr on Aug 8, 2018 | parent | prev | next [–]

...including error handling in async worker loops, please.

greyman on Aug 8, 2018 | prev | next [–]

Python is my language of first choice, but I must say that I am not that thrilled how this multithreading ended up. There are many tutorials about the topic promising to explain how it works, usually in the form of "simple introduction". But when one tries to implement something production-ready, with correct error handling etc., things starts to complicate pretty quickly; at least that was my experience. I don't want to accuse anyone specifically, but most of the tutorials I saw seems to portrait it in a way, that it looks easier than it actually is.

Ultimately, my company decided, that instead of fighting with asyncio, certain projects will switch to Go.

sametmax on Aug 8, 2018 | parent | next [–]

That's because most of those tutorials have not been written by somebody actually putting something in production.

I've been using asyncio for a while now, and you can't get away with a short introduction since:

- it's very low level

- it's full of design flaws and already has accumulated technical debt

- it requires very specific best practices to be usable

I'm not going to write a tutorial here, it would take me a few days to make a proper one, but a few pointers nobody tells you:

- asyncio solves one problem, and one problem only: when the bottleneck of your program is network IO. It's a very small domain. Most programs don't need asyncio at all. Actually many programs with a lot of network IO don't have performance problems, and hence don't need asyncio. Don't use asyncio if you don't need it: it adds complexity that is worth it only if it solves your problem.

- asyncio is mostly very low level. Unless you code your own lib or framework with it, you probably don't want to use it directly. E.G: if you want to make http requests, use aiohttp.

- use asyncio.run_until_complete(), not asyncio.run_forever(). The former will crash on any exception, making debugging easy. The later will just display the stack trace in the console.

- talking about easy debugging, activate the various debug features when not in prod (https://docs.python.org/3/library/asyncio-dev.html#debug-mod...). Too many people code with asyncio in the dark, and don't know there are plenty of debug info available.

- await is just a way to inline a callback. When you do "await", you say 'do the stuff', and any lines of code that are after "await" are called when "await" is done. You can run asynchronous things without "await". "await" is just useful if you want 2 asynchronous things to happen one __after__ another. Hence, don't use it if you wants 2 asynchronous things to progress in parallel.

- if you want to run one asynchronous thing, but not "await" it, call "asyncio.ensure_future()".

- errors in "await" can be just caught with try/except. If you used ensure_future() and no "await", you'll have to attach a callback with "add_done_callback()" and check manually if the future has an exception. Yes, it sucks.

- if you want to run one blocking thing, call "loop.run_in_executor()". Careful, the signature is weird.

- CPU intensive code blocks the event loop. loop.run_in_executor() use threads by default, hence it doesn't protect you from that. If you have CPU intensive code, like zipping a lot of files or calculating your own precious fibonacci, create a "ProcessPoolExecutor" and use run_in_executor() with it.

- don't use asyncio before Python 3.5.3. There is a incredibly major bug with "asyncio.get_event_loop()" that makes it unusable for anything that involve mixing threads and loops. Yep. Not a joke.

- but really use 3.6. TCP_NODELAY is on by default and you have f-string anyway.

- don't pass the loop around. Use asyncio.get_event_loop(). This way your code will be independent of the loop creation process.

- you do pretty much nothing yourself in asyncio. Any async magic is deep, deep down the lib. What you do is define coroutines calling the magic things with ensure_future() and await. Pretty much nothing in your own code is doing IO, it's just asking the asyncio code to do IO in a certain order.

- you see people in tutorials simulate IO by doing "asyncio.sleep()". It's because it's the easiest way to make the event loop switch context without using the network. It doesn't mean anything, it just pauses and switch, but if you see that in a tutorial, you can mentally replace it with, say, an http call, to get a more realistic picture.

- asyncio comes with a lot of concepts, let's take a time to define them:

```
    * Future: an object with a thing to execute, with potentially some callbacks to be called after it's
executed.

    * Task: a subclass of future. The thing to execute is a coroutine,, and the coroutine is immediately
scheduled in the event loop when the task is instantiated. When you do ensure_future(coroutine), it returns
a Task.

    * coroutine: a generator with some syntaxic sugar. Honestly that's pretty much it. They don't do much by
themself, except you can use await in them, which is handy. You get one by calling a coroutine function.

    * coroutine function: a function declared with "async def". When you call it, it doesn't run the code of
the function. Instead, it returns a coroutine.

    * awaitable: any object with an __await__ method. This method is what the event loop uses to execute
asynchronously the code. coroutines, tasks and futures are awaitables. Now the dirty secret is this: you can
write an __await__ method, but in it, you will mostly call the __await__ from some magical object from deep
inside asyncio. Unless you write a framework, don't think too much about it: awaitable = stuff you can pass
to ensure_future() to tell the event loop to run it. Also, you can "await" any awaitable.

    * event loop: the magic "while True" loop that takes awaitables, and execute them. When the code hits
"await", the event loop switch from one awaitable to another, and then go back to it later.

    * executor: an object that takes code, execute it in a __different__ context, and return a future you
can await in your __current__ context. You will use them to run stuff in threads or separate processes, but
magically await the result in your current code like it's regular asyncio. It's very handy to naturally
integrate blocking code in your workflow.

    * event loop policy: the stuff that creates the loop. You can override that if you are writing a
framework and wants to get fancy with the loop. Don't do it. I've done it. Don't.

    * task factory: the stuff that creates the tasks. You can override that if you are writing a framework
and wants to get fancy with the tasks. Don't do it either.

    * protocols: abstract class you can implement to tell asyncio __what__ to do when it establish/loose a
connection or send/receive a packet. asyncio instantiate one protocol for each connection. Problem is: you
```

```
can't use "await" in protocols, only old fashion callback.

    * transports: abstract class you can implement to tell asyncio __how__ to establish/loose a connection
or send/receive a packet.
```

Now, I'm putting the last point separately because if there is one thing you need to remember it's this. It's the most underrated secret rules of asyncio. The stuff that is literally written nowhere ever, not in the doc, not in any tuto, etc.

asyncio.gather() is the most important function in asyncio
==========

You see, everytime you do asyncio.ensure_future() or loop.run_in_executor(), you actually do the equivalent of a GO TO. (see: https://vorpus.org/blog/notes-on-structured-concurrency-or-g...)

You have no freaking idea of when the code will start or end execution.

To stay sane, you should never, ever, have an dangling awaitable anywhere. Always get a reference on all your awaitables. Decide where in the code you think their life should end.

And at this very point, call asyncio.gather(). It will block until all awaitables are done.

E.G, don't:

```
asyncio.ensure_future(bar())
asyncio.get_event_loop().run_in_executor(None, barz)
await asyncio.sleep(10)
```

E.G, do:

```
foo = asyncio.ensure_future(bar())
fooz = asyncio.get_event_loop().run_in_executor(None, barz)
await asyncio.sleep(10)
await asyncio.gather(foo, fooz)  # this is The Only True Way
```

Your code should be a meticulous tree of hierarchical calls to asyncio.gather() that delimitates where things are supposed to stop. And if you think it's annoying, wait for debugging something which life cycle you don't have control over.

Of course it's getting old pretty fast, so you may want to write some abstraction layer such as https://github.com/Tygs/ayo. But I wouldn't use this one in production just yet.

dataflow on Aug 8, 2018 | root | parent | next [–]

Awesome comment. One thing I want to point out to those reading is that the nursery thing is an instantiation of the more general principle of, *if you're finding your code is getting convoluted, it's likely that you're missing a noun*. I can't explain this as well as others have, so see this comment: https://news.ycombinator.com/item?id=16468796

sametmax on Aug 8, 2018 | root | parent | next [–]

I just love this comment.

I'm going to steal it for my next training on how to design an API.

When you are a computer scientist, you want to think about your data structures so badly first. It fits your brain so well, and it's easier to understand a program from them than the rest of the code.

But it's a trap.

dataflow on Aug 8, 2018 | root | parent | next [–]

Yes, PLEASE do!! :) I've been *dying* myself to get chances to teach these kinds of ideas! Hardly anyone seems to teach this kind of thoughtful analysis. Eric Lippert deserves an enormous amount of credit for writing this series in particular -- trying to explain these ideas coherently has been a massive struggle for me, let alone trickling them down to a small example that's easy to digest. He's a really awesome guy I look up to... I've learned so much from his writing (this is only one example of many).

mtrovo on Aug 8, 2018 | root | parent | prev | next [–]

Wow, really nice list, I wish I knew it before I started to work with asyncio.

> stay sane, you should never, ever, have an dangling awaitable anywhere. Always get a reference on all your awaitables. Decide where in the code you think their life should end.

This is the most difficult part for me, it's not trivial to know if a function you're calling is async or not without looking at the function source, specially when you're using external libraries. Also by default there's no logs

about this kind of situation so it's a easy way to shoot yourself in the foot and waste 10 minutes debugging to find a dangling awaitable on a function call you didn't realize was async.

wruza on Aug 8, 2018 | root | parent | next [–]

And still people vote for async-await because “true light threads are hard to implement at low level”. This generator-based madness has to end, but few seem to understand what hassle it brings to their coding and what an alternative could be. I don’t get it.

sametmax on Aug 8, 2018 | root | parent | prev | next [–]

That's why you should activate the debug features I mentioned. It will write in the console when you are calling an async thingy without getting the result from it.

takeda on Aug 8, 2018 | root | parent | prev | next [–]

Anything that's defined as "async def" and that you call with await and friends should be async.

Yes, it's possible to write coroutine and use "async def" without any await inside, but in those cases the library authors should just made it a normal function.

I would say that this is a bug in the library.

multani on Aug 8, 2018 | root | parent | prev | next [–]

> - don't pass the loop around. Use asyncio.get_event_loop(). This way your code will be independent of the loop creation process.

Eh. I've been passing the loop around as an optional kw argument in most of my code...

The idea was for the code not to depend on a global somewhere (I hate globals) and to "be sure" the loop used among all the code was the same, unless explicitly passed. Of course I never used that "feature". I thought I read this somewhere when I was looking up at Twisted and they were saying to pass it explicitly, but I'm not so sure now...

takeda on Aug 8, 2018 | root | parent | next [–]

You supposed to have only a single event loop per thread, the standard event loop policy ensures that the value is thread local (you can change that by modifying the policy), unless you're doing something unusual with multiple loops in the same thread you will never need to pass the value.

Also if you are passing the loop and are doing multi threading, you need to be careful, because if you pass it to another thread you might see weird issues.

I initially also started explicitly pass loop around but once decided to combine asyncio with threads I realized that it is better to trust get_event_loop() to do the job correctly. The only exception is when I need to schedule coroutine in one thread for another thread. In that case I need loop from a different thread so I can invoke call_soon_threadsafe().

stinos on Aug 8, 2018 | root | parent | prev | next [–]

one problem only: when the bottleneck of your program is network IO

Do you mean literally what this says, or are you rather using 'network IO' as some (extremely) abstract term for any type of communication? Just checking because I haven't used asynchronous programming in Python but did so in other languages and we do things like await hardwareAxis.GoToTargetPosition(position=50, veolcity=100). Not what most people think of when reading network IO, that one.

sametmax on Aug 8, 2018 | root | parent | next [–]

While async / await, futures, and the event loop are generic mechanisms, the asyncio module itself only implement selectors for sockets (ok, and subprocess pipes). You can't even do async file system operations with it: you need to call run_in_executor().

Now that doesn't mean you could not implement a selector that does asynchronous UI IO and plug it to the event loop. But the asyncio module doesn't provide it right now, and no lib that I know of does it either.

takeda on Aug 8, 2018 | root | parent | prev | next [–]

Good information, but it all depends on use case, for example I use a lot of await and "async with" in coroutines.

Then start tasks as:

```
tasks = [coroutine(i) for i in parameters]
```

and then iterate over results using

```
for task in asyncio.as_completed(tasks)
```

You can also start threads and then dispatch coroutines to them.

There are many ways of using it.

metalliqaz on Aug 8, 2018 | root | parent | prev | next [–]

I wish I could favorite comments on HN

chatmasta on Aug 8, 2018 | root | parent | next [–]

You can. Click on the timestamp and then favorite it.

sachin18590 on Aug 8, 2018 | parent | prev | next [–]

I am curious what considerations your company had before switching some projects to Go. Python multithreading has been an issue for us as well. While asyncio looked good in the tutorials, gevent was much easier to work with. However, we still face multiple issues moving our celery workers to gevent and I am not sure if there is a better production friendly alternative for celery-gevent in python.

greyman on Aug 8, 2018 | root | parent | next [–]

I am actually not that familiar with that decision, since I still work on python projects (but those don't require multithreading). But the guys who work on Go projects mostly cited the following advantages: 1) good performance, since it is compiled, 2) easier deployment, since it compiles into single statically-linked file, and 3) multithreading is backed into the language.

Instead of gevent, I had quite a good experience with concurrent.futures; but I used it only for simple things like download multiple URLs in parallel, etc. Anyway, I can't help, but in retrospective all this multithreading looks to me a bit like being hacked into python language as an afterthought.

sandGorgon on Aug 8, 2018 | root | parent | prev | next [–]

we love gevent as well. we use rq instead of celery . Try that instead (http://python-rq.org/docs/workers/)

sametmax on Aug 8, 2018 | root | parent | next [–]

Rq makes task queues easy again.

But I feel like celery is mostly badly documented: it show off the complexity of it instead of what can be simple.

E.G: did you know you can use celery without any broker ? None. Not even redis.

https://www.python-celery.com/2018/07/03/simple-celery-setup...

sachin18590 on Aug 10, 2018 | root | parent | prev | next [–]

We tried rq but wanted something more configurable/extensible. Also, worker level concurrencies were easier to manage with celery. But that said, rq definitely is a much easier and lightweight alternative to celery and is my goto option whenever I need to spin up something quick without much overhead.

kofejnik on Aug 8, 2018 | root | parent | prev | next [–]

+1 for RQ, it's so much cleaner

blattimwind on Aug 8, 2018 | parent | prev | next [–]

> not that thrilled how this multithreading ended up

asyncio isn't multithreading at all.

apoorvgarg on Aug 8, 2018 | parent | prev | next [–]

Thanks for the perspective (especially for me, since Go is my primary language.)

Just to clarify, the primary target audience for the article is anyone who is getting started with asyncio (although I know of people who have been using asyncio, but don't really understand whats going on).

tlrobinson on Aug 8, 2018 | prev | next [–]

This is essentially how modern JavaScript works, in particular with the addition of async/await syntax [1] (which was originally from C#, I think), but it's been possible with libraries like task.js, co, and Bluebird [2] since generator functions

were available (either natively or via transpiling).

The main difference is in JavaScript the event loop is automatic and hidden, and asynchronous IO is the default, so it's a bit harder to shoot yourself in the foot.

1. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Refe...
2. https://github.com/mozilla/task.js https://github.com/tj/co http://bluebirdjs.com/docs/api/promise.coroutine.html

heavenlyblue on Aug 8, 2018 | parent | next [2 more]

sadgit on Aug 8, 2018 | prev | next [–]

I'm still bummed that Python took this direction. Maybe introducing new keywords into the language for event loop concurrency was Python's way of satisfying "explicit is better than implicit" but i can's shake the feeling that callback passing and generator coroutines are a fad that is complex enough to occupy the imagination of a generation of programmers while offering little benefit compared to green threads.

nerdwaller on Aug 8, 2018 | parent | next [–]

Having worked in asyncio for a bit I don't entirely follow this (it truly could just be familiarity), very little of asyncio (especially post `async/await` were introduced in the language) is callback based and reads more procedural.

Regarding generator coroutines it feels like a natural evolution of the language. Given that yield previously suspended the current function's state providing value(s) to the closure, it only makes sense that yield (on the producer side)/await (on the consumer side) does the same thing but in an event loop based context.

I can't speak deeply enough about green threads, but from my understanding there's much less magic (as you cite "explicit") in an async/await world vs the magic ("implicit") world of green threads.

```
async def thing():
    print('before')
    await asyncio.sleep(0)
    print('after')
```

Vs

```
def thing():
    print('before')
    gevent.sleep(0)
    print('after')
```

There's nothing clear in the latter when something yields or otherwise passes control flow.

Having worked in a few evented systems, I find the explicit shift to the runtime is valuable.

BlackFly on Aug 8, 2018 | root | parent | next [–]

The cooperating part of this concurrency model is the complicated part. Consider how you would go about making an orm like sqlalchemy cooperate. Now you have to access properties like this:

```
name = await account.user.name
```

since a lookup may have to occur. This is extremely unnatural and would be better if you could just avoid writing await yet still depend on it being concurrent without blocking your event loop. The fact that the caller needs to understand that the callee supports this form of concurrency is an abstraction inversion in my opinion. Python forces this concurrency to be explicit, but it would be more powerful and more natural if it were implicit:

```
name = account.user.name
```

iamforreal on Aug 8, 2018 | root | parent | next [–]

I've gone back and forth on this so much.

On the one hand, it's really annoying when your client library doesn't actually support asyncio compatible code (ex libraries which perform synchronous network or disk reads/writes), and you have to wrap everything in an executor.

On the other hand, making it explicit ensures I'm actually doing things async. "Leaf" functions with an async containing no await is now a red flag to me.

It's a mental tax to remember that I may actually be returning a future instead of the result of a future (similar to how you can return a function but not the result of that function being executed, or a non materialized generator), and having to call 'await x' instead of just assigning x kind of violates 'do what I mean'. In the end, async is (relatively) difficult, so I appreciate the enforced explicitness.

nerdwaller on Aug 12, 2018 | root | parent | prev | next [–]

This is certainly one "drawback", depending on your perspective, and certainly a cost more of an ORM and how they tend to work than a runtime environment. You'd have a similar issue, for example, in Go if you want to lazy load a property (however there you can't await a goroutine).

Long story short, this drawback tends to be primarily based on experience of the overall system. Coming from traditional rails/django/etc will make these constructs seem awkward.

nicoburns on Aug 8, 2018 | root | parent | prev | next [–]

I think this just means that a sqlalchemy style ORM doesn't fit the model. If you had an ORM where the calls which could call cause database queries were distinct from calls which just looked up local properties, then this would work fine...

blattimwind on Aug 8, 2018 | root | parent | next [–]

I think it mostly means that identity-mapped objects which may be expired aren't really compatible. Of course, one could always

```
await session.commit()
user.name  # BlahError: Object not loaded

# correct
await session.commit()
await user.refresh()
user.name
```

This might actually make people more actively avoid SELECT n+1, since lazy-loading would error out by default or require an extra await.

Another thing that might not be completely obvious, but sessions and their objects (session×objects = transaction state) are never shared between threads, similarly it would be unwise to share them between different asynchronous tasks.

sametmax on Aug 8, 2018 | parent | prev | next [–]

It's doesn't have to be complex though.

It's complex because the asyncio API is terrible. It exposes loop/task factory and life cycle way to much, and shows it off in docs and tutorials.

Hell, we had to wait for 3.7 to get asyncio.run() !

Even the bridge with threads, which is a fantastic feature, has a weird API:

```
await = asyncio.get_event_loop().run_in_executor(None, callback)
```

Also, tutorials and docs give terrible advices. They tell you to run_forever() instead of run_until_complete() and forget about telling you to activate debug mode. They also completly ignore the most important function of all: asyncio.gather().

asyncio can become a great thing, all the foundational concepts are good. In it's current form, though, it's terrible.

What we need is a better API and better doc.

A lot of people are currently understanding this and trying to fix it.

Nathaniel J. Smith is creating trio, a much simpler, saner alternative to asyncio: https://github.com/python-trio/trio

Yury Selivanov is fixing the stdlib, and experiments with better concepts on uvloop first to integrated them later. E.G: Python 3.8 should have trio's nurseries integrated in stdlib.

Personally, I don't want to wait for 3.8, and I certainly don't want the ecosystem to be fragmented between asyncio, trio or even curio. We already had the problem with twisted, tornado and gevent before.

So I'm working on syntaxic sugar on top of asyncio: https://github.com/Tygs/ayo

The goal is to make the API clean, easy to use, and that enforced the best practices, but stay 100% compatible with asyncio (it uses it everywhere) and it's ecosystem so that we don't get yet-another-island.

It's very much a work in progress, but I think it demonstrate the main idea: asyncio is pretty good already, it just needs a little love.

sebcat on Aug 8, 2018 | parent | prev | next [–]

> i can’s shake the feeling that callback passing and generator coroutines are a fad [...]

Callback passing and coroutines are well known techniques that's been around for a while. Generators are just coroutines that yield to their parent. I remember using these concepts in C and Tcl ~15 years ago[1] and they were well known then. According to wikipedia (citing Knuth), the term coroutine was coined in the late 50's by Conway.

Callback passing and coroutines suits some problems well. Sure there are situations where they do not fit and if people use a hammer for all of their problems they will create new ones. I wouldn't call it a fad, though the techniques may be a bit hyped up in some circles.

[1] I don't remember when Tcl got the coroutine package, that may have been later

carreau on Aug 8, 2018 | parent | prev | next [–]

Note that the asycn/await syntax, coroutines, an asyncio are 3 independents part. If you do not like callback and Future have a look at trio[1] that takes a quite different approach.

http://trio.readthedocs.io/en/latest/

cshenton on Aug 8, 2018 | parent | prev | next [–]

For me it had the opposite effect.

Working with async-await syntax was the last straw that made me finally go "there's got to be a better way" and find a language can handle concurrency without the semantic overhead (in my case Go, but there are others).

kjeetgill on Aug 8, 2018 | parent | prev | next [–]

I agree it's always left a bag taste in my mouth. I loved generators and yield/yield from, but I was stuck on 2.7 for a long time so I never quite understood the motivation for async/await over them.

One issue is that it "reifys" the "colored function" problem that green threads like goroutines don't have!

Side note: Java world is working on green threads/fibers for the JVM in Project Loom.

[0]: http://cr.openjdk.java.net/~rpressler/loom/Loom-Proposal.htm...

jrs95 on Aug 8, 2018 | parent | prev | next [–]

I'm not an expert on this but it seems like they just didn't want to find a way to give you green threads without GIL. This had already been done in another Python implementation: https://en.m.wikipedia.org/wiki/Stackless_Python

Stackless has a proven model. Basically the same model as goroutines and channels. It's the reason EVE Online is able to run its primary game server in Python with such a large number of users.

azeirah on Aug 8, 2018 | parent | prev | next [–]

Perhaps asyncio is just a bit more low-level than we're used to in Python. Maybe we'll end up with something analogous to the "requests" library, but then for asyncio...

shawn on Aug 8, 2018 | parent | prev | next [–]

Effing thank you. I don't think most people realize just how convenient green threads are. It kills me to see devs stuck in the local maximum of callback hell.

That sad, https://www.usenix.org/system/files/conference/atc12/atc12-f... raises some interesting points in defense of one-way RPC. The key is not to allow returns.

kbaker on Aug 8, 2018 | root | parent | next [–]

Isn't Python's async/await syntax an implementation of green threads? I mean using await is almost exactly the cooperative scheduling idea. The article may use Futures and callbacks but you can just as easily do something like:

```
result = await fake_network_request('one')
```

toast0 on Aug 8, 2018 | root | parent | next [–]

They're sort of similar, and you can probably get the same work done in either system, but I think real threading (green or otherwise), may leave you with less cognitive load. Spawning a thread may be complex, and thinking about how the threads are scheduled is often complex, but what each thread does can be very simple -- and you don't have to think about 'long running things need to be futured/awaited', you just do things in a straightforward way in the thread (caveat: slightly less straightforward if you need thread actions to be cancellable).

Green threads may be running an event loop underneath, but it's a useful abstraction in many contexts.

pdkl95 on Aug 8, 2018 | root | parent | next [–]

> ... you don't have to think about 'long running things need to be futured/awaited', you just do things in a straightforward way in the thread

https://www.youtube.com/watch?v=bzkRVzciAZg

Six years later, very little has change in the arguments about events vs threads.

pas on Aug 8, 2018 | root | parent | prev | next [–]

> Spawning a thread may be complex, and thinking about how the threads are scheduled is often complex, but what each thread does can be very simple

And that's how it starts, and in the end it's New Year's Eve and you're somehow, again, debugging a deadlock.

> green threads

yes please

gpderetta on Aug 8, 2018 | root | parent | next [–]

You can deadlock with futures as well. Except that with futures you do not get a (two actually) nice call stack pointing to the deadlocked resource.

yen223 on Aug 8, 2018 | root | parent | prev | next [–]

Python has proper threads, and they're anything but simple.

toast0 on Aug 9, 2018 | root | parent | next [–]

Python threads aren't simple, because of the shared everything model python uses, so any variable access requires the GIL. Shared nothing threads are much simpler to work with.

manaskarekar on Aug 8, 2018 | parent | prev | next [–]

This might be interesting reading. Comments on rust's approach to async from green thread.

https://aturon.github.io/blog/2016/08/11/futures/

nemothekid on Aug 8, 2018 | root | parent | next [–]

One of the key reasons Rust took this approach is because Rust's implementation is zero cost - it doesn't require a runtime to implement. It is potentially very efficient, and thanks to Rust's other guarantees its very safe to use. It's as close to bare metal as you can get for a async framework and as a result its incredibly efficient.

However, to me, the implementation is a lot complex that green threads. The Future's crate has had a lot of churn, and when I first dabbled in it a while back, it was one of the first few times I struggled to understand what the compiler errors even meant as the types were so deep. Compared to golang's 'go', futures are harder to understand, plus you have to rewrite all your networking/blocking code to be compatible (Python would still likely have to do the same, but I think it could be done in such a way that if you were using the system provided networking libraries, you could get compatibility for "free").

Python doesn't benefit from the Rust benefits explained in that article. Python already has a garbage collector and runtime. Python is single threaded.

I don't follow the Python language to have a well informed opinion on why they went with futures, but I doubt its close to the reasoning that Rust chose.

smittywerben on Aug 10, 2018 | root | parent | next [–]

> Python is single threaded.

That's not true, there are multiple threads in Python i.e. zlib from the standard library.

People want unsafe code to communicate concurrently through the Python thread state. I'll let someone else tackle that one.

steveklabnik on Aug 8, 2018 | root | parent | prev | next [–]

And we're going further, with async/await http://aturon.github.io/2018/04/24/async-borrowing/

protonfish on Aug 8, 2018 | parent | prev | next [–]

I wish they had implemented parallelism based on the actor model. It seems like the perfect high-level abstraction for managing parallel processes. All of the asyncio stuff feels too fine-grained for Python-style development.

shoo on Aug 8, 2018 | prev | next [–]

quoting the article:

> Concurrency is like having two threads running on a single core CPU.

> Parallelism is like having two threads running simultaneously on different cores

> It is important to note that parallelism implies concurrency but not the other way round.

Aurgh! I don't think this attempted definition-by-simile is helpful, or even somewhat correct.

I much prefer yosefk's way of framing things:

> > concurrent (noun): Archaic. a rival or competitor.

> > Two lines that do not intersect are called parallel lines.

...

> Computation vs event handling

> With event handling systems such as vending machines, telephony, web servers and banks, concurrency is inherent to the problem – you must resolve inevitable conflicts between unpredictable requests. Parallelism is a part of the solution - it speeds things up, but the root of the problem is concurrency.

> With computational systems such as gift boxes, graphics, computer vision and scientific computing, concurrency is not a part of the problem – you compute an output from inputs known in advance, without any external events. Parallelism is where the problems start – it speeds things up, but it can introduce bugs.

...

> concurrency is dealing with inevitable timing-related conflicts, parallelism is avoiding unnecessary conflicts

yosefk's whole essay about this is great: https://yosefk.com/blog/parallelism-and-concurrency-need-dif...

foxes on Aug 8, 2018 | parent | next [–]

I also initially thought the same thing. Page two of "Parallel and Concurrent programming in Haskell" maybe says it in a nicer way:

>A parallel program is one that uses a multiplicity of computational hardware ....

>concurrency is a program-structuring technique in which there are multiple threads of control...

(a pdf can readily be found with your favorite search engine for the full extract :)).

I would much prefer to see a precise, rigorous definition and then examples (or eg and then defn is also acceptable), instead of just a list of examples. Examples help you understand a rigorous statement. But, if you only give a hand waving explanation for something, I think it just creates more confusion in the end, as you never know exactly what is correct. It's leaving it open for ambiguity.

chatmasta on Aug 8, 2018 | parent | prev | next [–]

Also a big fan of this "Visualizing Concurrency in Go": http://divan.github.io/posts/go_concurrency_visualize/

meken on Aug 8, 2018 | prev | next [–]

I really liked this article. It's by far the most concise explanation of asyncio in python that I've come across. Also, great use of little "quoted" statements throughout that encourage you to stop and really understand what was said before moving on (these probably have a special name).

Bravo!

eliasson on Aug 8, 2018 | prev | next [–]

A few years ago I wrote an BitTorrent client in Python 3.5 to get to know asyncio better.

Maybe those blogposts are still of use to somebody:

- http://markuseliasson.se/article/introduction-to-asyncio/

- http://markuseliasson.se/article/bittorrent-in-python/

ioquatix on Aug 8, 2018 | prev | next [–]

Here is a comparison of `asyncio` (Python), `async` (Ruby) and Go: https://github.com/socketry/async-await/tree/master/examples...

I wrote a similar article but for Ruby: https://www.codeotaku.com/journal/2018-06/asynchronous-ruby/...

Yes, it's a good model for many use cases.

One thing I wondered about Ruby, is it really necessary to have the `await` keyword?

kasbah on Aug 8, 2018 | parent | next [–]

`await` is useful since with the absence of it you can schedule multiple tasks concurrently. In JS:

```
var task1 = someAsyncTask1()
var task2 = someAsyncTask2()

await Promise.all([task1, task2])
```

If `await` was implicit then task2 would wait for task1 to finish.

jrs95 on Aug 8, 2018 | prev | next [–]

It seems odd that Python ended up with asyncio when they had a clear and successful model they could’ve adopted from Stackless. It would have been more difficult to implement, but it would allow for the same benefits without requiring an asynchronous programming model, which would have reduced the total amount of effort involved in getting to having good concurrency in Python.

_Codemonkeyism on Aug 8, 2018 | prev | next [–]

Wasn't there a language where every call was async? Instead of async ... A/returing Future[A] it did/would return A from method calls.

If it didn't exist, one can imagine one.

A.x = 3 would be wrapped in A.map(_.x = 3) etc. So you write code that would be executed when you finally await a value. No more red/blue world. Would probably need coroutines instead of threads for executing.

marcosdumay on Aug 8, 2018 | parent | next [–]

Just to post it as an answer instead of a question. That's Haskell's IO.

It is just one of the lots of concurrency behaviors available in libraries. Also, parallelism is "free" on pure code.

_Codemonkeyism on Aug 9, 2018 | root | parent | next [–]

From my understanding, this is not Haskells IO - though my time with Haskell is limited.

1. Haskell uses special notation 'do' to handle access to IO wrapped values, e.g. (contrieved example, one would not use do for such simple cases)

```
y = do x <- xWithIO
    return x + 1
```

instead of

```
y = x + 1
```

2. Haskell method signatures do include IO, e.g.

```
doSomething   :: Int -> IO Int
```

instead of

```
def doSomething(i:Int):Int
```

3. Because IO is usually not the only effect managaging monad, as I've said in another comment, the type signature usually uses a type alias that does alias a monad transformer stack like

```
type Result a = ReaderT Env ( ErrorT  String ( StateT  Integer Identity))
```

or concurrency mixed in

4. This the same as my Scala code, where I have cats FutureT monad transformers with Scalactic errors OrT Every stacks showing up all over my APIs as a type alias of 'WithErrors'.

5. 'Also, parallelism is "free" on pure code.' Not sure what's that got to do with it, but yes if you have no concurrency problems (concurrent writes to shared data) you don't need to think about concurrency and parallelism is free.

But if my understanding is wrong, I'm happy to learn something about concurrency in Haskell without it showing in code and type signatures.

marcosdumay on Aug 9, 2018 | root | parent | next [–]

Well, it does not have the exact same syntax of your example. Even more because your example was pure. Haskell does that automatically for pure code too (`y = x + 1` would do exactly what you described) but it's not really relevant.

IO code always returns a promise, and the next statement on a `do` block may await the previous promise and yield the execution to whatever other piece of code can run, based on some rules on the compiler, based in large part on data dependency. If I'm reading your comment correctly, that is what you are asking for.

_Codemonkeyism on Aug 10, 2018 | root | parent | next [–]

Sorry for being so confusing

"Even more because your example was pure."

No it wasn't which is the whole point.

```
y = x + 1
```

means

```
y = x.map(_ + 1)
```

by default with async effects.

"same syntax"

Which is also the point, that there is syntax for the effects. When everything is async, there should be no special syntax as you have that syntax all over your code and it's redundant.

icebraining on Aug 8, 2018 | parent | prev | next [–]

Isn't Haskell somewhat like that, due to being lazy by default?

_Codemonkeyism on Aug 8, 2018 | root | parent | next [–]

In Haskell you'd have the type signature everywhere I think, mostly as a monad transformer.

jbarham on Aug 8, 2018 | prev | next [–]

Someone send this article to Armin Ronacher (creator of Flask, Jinja, etc.) so he can understand asyncio since he wrote a much longer and more detailed article explaining why he doesn't understand asyncio (previous discussion at https://news.ycombinator.com/item?id=12829759).

kyberias on Aug 8, 2018 | prev | next [–]

Is this any different than in C#?

mikec3010 on Aug 8, 2018 | prev | next [–]

My first endeavor with asyncio felt worse than a beating with a wet rubber hose. But it was a character building experience to really get up close and personal with the asynchronous model and I can definitely see its advantages over imperative, as well as it's always good to have another tool in the box.

anc84 on Aug 8, 2018 | prev [–]

Thinly veiled advertisement for a "Intelligent Infrastructure Analytics - a Machine Learning driven approach for DevOps & SREs of modern age" company. Seems like hackernoon just published a native advertisement? Not surprisingly shady though, considering the "buy crypto with credit card" link in the top bar.

Guidelines | FAQ | Lists | API | Security | Legal | Apply to YC | Contact

Search: